

OPERATING SYSTEMS

(UNIT-III)

PROCESS SYNCHRONIZATION

Cooperating processes require an inter process communication (IPC) mechanism that will allow them to exchange data and information. There are two fundamental models of inter process communication: (1) shared memory and (2) message passing. In the shared-memory model, a region of memory that is shared by cooperating processes is established. Processes can then exchange information by reading and writing data to the shared region. In the message passing model, communication takes place by means of messages exchanged between the cooperating processes.

One solution to the producer-consumer problem uses shared memory. To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer. This buffer will reside in a region of memory that is shared

by the producer and consumer processes. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. Two types of buffers can be used. The unbounded buffer places no practical limit on the size of the buffer. The consumer may have to wait for new items, but the producer can always produce new items. The bounded buffer assumes a fixed buffer size. In this case, the consumer must wait if the buffer is empty, and the producer must wait if the buffer is full.

The shared buffer is implemented as a circular array with two logical pointers: in and out. The variable 'in' points to the next free position in the buffer; 'out' points to the first full position in the buffer. The buffer is empty when $in == out$; the buffer is full when $((in + 1) \% BUFFER_SIZE) == out$.

The producer process:

```
item nextProduced;
while (true) {
}
/* produce an item in nextProduced */
while ( ((in + 1) % BUFFER_SIZE) == out )
; /* do nothing */
buffer[in] = nextProduced;
in = (in + 1) % BUFFER_SIZE;
```

The consumer process:

```
item nextConsumed;
while (true) {
}
while (in == out)
; // do nothing
nextConsumed = buffer[out];
out = (out + 1) % BUFFER_SIZE;
/* consume the item in nextConsumed */
```

Suppose we want to modify the algorithm to remedy this deficiency. One possibility is to add an integer variable counter, initialized to 0. counter is incremented every time we add a new item to the

buffer and is decremented every time we remove one item from the buffer. The code for the producer process can be modified as follows:

```
while (true) {  
  
    /* produce an item in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; /* do nothing */  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```

The code for the consumer process can be modified as follows:

```
while (true) {  
  
    while (counter == 0)  
        ; /* do nothing */  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in nextConsumed */  
}
```

THE CRITICAL SECTION PROBLEM

Each process in a system has a segment of code called a **critical section** in which the process may be changing common variables, updating a table, writing a file etc. The important feature of the system is that when one process is executing in its critical section, no other process is to be allowed to execute in its critical section that is no two processes are executing in their critical sections at the same time. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (TRUE);
```

General structure of a typical process P_i .

A critical section is a piece of code that only one thread can execute at a time. If multiple threads try to enter a critical section, only one can run and the others will sleep.

A solution to the critical section problem must satisfy the following three requirements:

- ✚ Mutual exclusion: If a process is executing in critical section, then no other process can be executing in their critical section.
- ✚ Progress: If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its critical section next and this selection cannot be postponed indefinitely.
- ✚ Bounded waiting: There exists a bound or limit on the number of times that other processes are allowed to enter their critical section after a process has made a request to enter its critical section and before that request is granted.

PETERSON'S SOLUTION

A classic software based solution to the critical section problem is known as Peterson's solution. It provides mutual exclusion, progress and bounded waiting requirements.

Peterson's solution is restricted to two processes (P_i & P_j) that alternate execution between their critical sections and remainder sections. Peterson's solution requires two data items to be shared between the two processes:

```
int turn;  
boolean flag[2];
```

The variable **turn** indicates whose turn it is to enter its critical section.

If $\text{turn} = i$, then the process P_i is allowed to execute in its critical section.

The **flag** array is used to indicate if a process is ready to enter its critical section.

If $\text{flag}[i]$ is true, means that P_i is ready to enter its critical section.

To enter the critical section, P_i first sets $\text{flag}[i]$ to be true and then sets **turn** to the value j , thereby asserting that if the other process wants to enter the critical section. If both processes try to enter at the same time, **turn** will be set to both i and j at the same time.

The solution is:

```
do
{
    flag[i]=TRUE;
    turn=j;
    while(flag[j] && turn==j);

    // critical section

    flag[i]=FALSE;
    //remainder section
} while(TRUE);
```

The algorithm does satisfy the three essential criteria to solve the critical section problem. The three criteria are mutual exclusion, progress, and bounded waiting.

For two processes P0 and P1:

Mutual exclusion

P0 and P1 can never be in the critical section at the same time: If P0 is in its critical section, then flag [0] is true and either flag [1] is false (meaning P1 has left its critical section) or turn is 0 (meaning P1 is just now trying to enter the critical section, but graciously waiting). In both cases, P1 cannot be in critical section when P0 is in critical section.

Progress

A process cannot immediately re-enter the critical section if the other process has set its flag to say that it would like to enter its critical section.

Bounded waiting

In Peterson's Algorithm, a process will not wait longer than one turn for entrance to the critical section: After giving priority to the other process, this process will run to completion and set its flag to 0, thereby allowing the other process to enter the critical section.

MUTEX LOCKS

Operating-systems designers build software tools to solve the critical-section problem. The simplest of these tools is the mutex lock. (In fact, the term mutex is short for mutual exclusion.) We use the mutex lock to protect critical regions and thus prevent race conditions. That is, a process must acquire the lock before entering a critical section; it releases the lock when it exits the critical section. The `acquire()` function acquires the lock, and the `release()` function releases the lock.

A mutex lock has a Boolean variable available whose value indicates if the lock is available or not. If the lock is available, a call to `acquire()` succeeds, and the lock is then considered unavailable.

The definition of `acquire()` is as follows:

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false;;  
}
```

The definition of `release()` is as follows:

```
release() {  
    available = true;  
}
```

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (true);
```

Solution to the critical-section problem using mutex locks.

The main disadvantage of the implementation given here is that it requires busy waiting. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the call to `acquire()`.

SEMAPHORES

The various hardware based solutions to the critical section problem are complicated for application programmers to use. To overcome this difficulty, we can use a synchronization tool called a semaphore.

A semaphore S is an integer variable that is accessed only through standard atomic operations: `wait()` and `signal()`.

Wait:

```
Wait(S)
{
    While S<=0;
    S--;
}
```

Signal:

```
signal(S)
{
    S++;
}
```

Modifications to the integer value of the semaphore in the `wait()` and `signal()` operations must be executed indivisibly. When one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Usage

OS's distinguish between counting and binary semaphores. The value of a counting semaphore can range over an unrestricted domain. The value of a binary semaphore can range only between 0 and 1. Binary semaphores are known as **mutex locks** as they are locks that provide mutual exclusion.

```
do {
    waiting(mutex);

    // critical section

    signal(mutex);

    // remainder section
}while (TRUE);
```

Binary semaphores are used to deal with the critical section problem for multiple processes. Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Each process that wishes to use a resource performs a `wait()` operation on the semaphore. When a process releases a resource, it performs a `signal()` operation. When count is 0, all the resources

are used. After that, a process that wants to use a resource will block until count becomes greater than 0.

Implementation

The main disadvantage of the semaphore is that it requires **busy waiting**. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. Busy waiting wastes CPU cycles that some other process might be able to use productively. This type of semaphore is called a **spinlock** because the process spins while waiting for the lock.

To overcome, the need for busy waiting the definition of wait () and signal() semaphore operations can be modified. When a process executes the wait() operation and finds that the semaphore value is not positive, it must wait. Rather than engaging in busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore and the state of the process is switched to the waiting state. Then control is transferred to CPU scheduler which selects another process to execute.

A process that is blocked waiting on a semaphore S, should be restarted when some other process executes a signal() operation. The process is restarted by a wakeup() operation which changes the process from the waiting state to the ready state. Process is then placed in the ready queue.

The semaphore in C language as:

```
typedef struct
{
    int value;
    struct process *list;
} semaphore;
```

Each semaphore has an integer **value** and a **list** of processes list. When a process must wait on a semaphore, it is added to the list of processes. A signal () operation removes one process from the list of waiting processes and awakens that process.

The **wait()** semaphore operation is defined as:

```
wait(semaphore *S)
{
    S->value --;
    if(S->value<0)
    {
        add this process to S-> list;
        block();
    }
}
```

The **signal()** semaphore operation is defined as:

```
Signal(semaphore *S)
{
    S->value++;
    if(S->value <= 0)
    {
        Remove a process P from S->list;
        wakeup();
    }
}
```

Deadlocks and Starvation

The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event (execution of a **signal()**) that can be caused only by one of the waiting processes. When such a state is reached, these processes are said to be **deadlocked**.

Consider two processes P0 and P1, each accessing two semaphores S and Q. Then –

<u>P0</u>	<u>P1</u>
wait(S)	wait(Q)
wait(Q)	wait(S)
-----	-----
-----	-----
signal(S)	signal(Q)
signal(Q)	signal(S)

suppose that P0 executes wait(S) and the P1 executes wait(Q). when P0 executes wait(Q), it must wait until P1 executes signal(Q). similarly, when P1 executes wait(S), it must wait until P0 executes signal(S). since these two signal() operations cannot be executed, P0 and P1 are deadlocked.

CLASSIC PROBLEMS OF SYNCHRONIZATION

These synchronization problems are examples of large class of concurrency control problems. In solutions to these problems, we use semaphores for synchronization.

The Bounded Buffer problem

Here the pool consists of **n** buffers, each capable of holding one item. The **mutex** semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The **empty** and **full** semaphores count the number of empty and full buffers. The semaphore **empty** is initialized to the value **n**, the semaphore **full** is initialized to value **0**.

The code below can be interpreted as the producer producing full buffers for the consumer or as the consumer producing empty buffers for the producer.

The structure of the producer process

```
do {  
    // produce an item in nextp  
    wait (empty);  
    wait (mutex);  
    // add the item to the buffer  
    signal (mutex);  
    signal (full);  
} while (TRUE);
```

The structure of the consumer process

```
do {  
    wait (full);  
    wait (mutex);  
    // remove an item from buffer to nextc  
    signal (mutex);  
    signal (empty);  
    // consume the item in nextc  
} while (TRUE);
```

The Readers – Writers Problem

A data base is to be shared among several concurrent processes. Some of these processes may want only to read the database (readers) whereas others may want to update the database (writers). If two readers access the shared data simultaneously, no adverse effects will result. If a writer and some other thread, access the database simultaneously, problems occur.

To prevent these problems, writers have exclusive access to the shared database. This synchronization problem is referred to as readers – writers problem.

The simplest readers writers problem requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object. No reader should wait for other readers to finish simply because a writer is waiting.

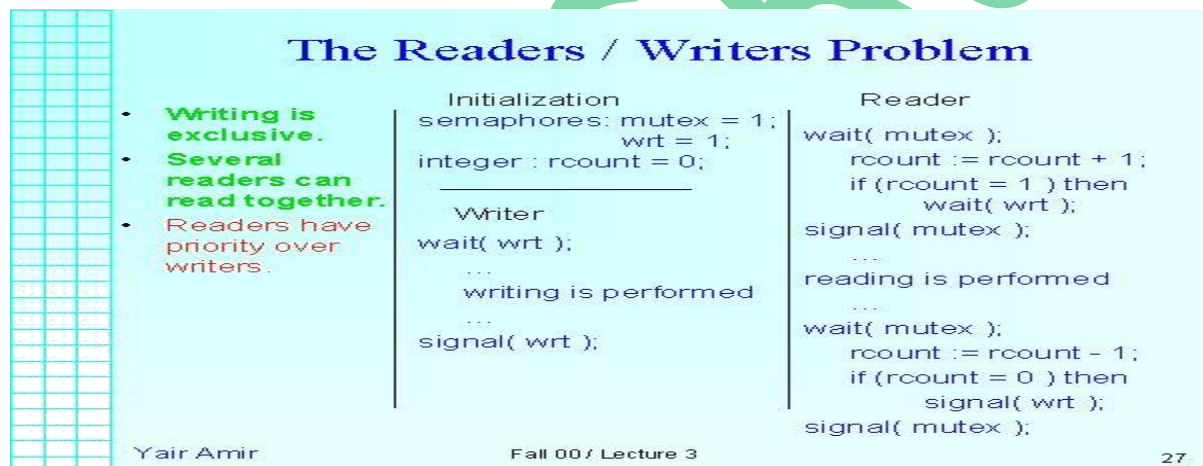
The second readers writers problem requires that once a writer is ready, that writer performs its write as soon as possible, that is if a writer is waiting to access the object, no new readers may start reading.

In the solution to the first readers – writers problem, the reader processes share the following data structures:

Semaphore mutex, wrt;

int readcount;

The structure of reader and writer processes are---



Semaphores **mutex** and **wrt** are initialized to 1, **readcount** is initialized to 0. Semaphore **wrt** is common to both reader and writer processes. The **mutex** semaphore is used to ensure mutual exclusion when the variable **readcount** is updated. The **readcount** variable keeps track of how many processes are currently reading the object. The semaphore **wrt** functions as a mutual exclusion semaphore for the writers. It is also used by the first or last reader that enters or exits the critical section. It is not used by readers who enter or exit while other readers are in their critical section.

Dining Philosophers Problem



Consider five philosophers who spend their lives thinking and eating. The philosophers share a circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chop sticks. When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her. A philosopher may pick up only one chopstick at a time. She cannot pick up a chopstick that is already in the hand of the neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chop sticks. When she is finished eating, she puts down both of her chop sticks and starts thinking again.

The dining philosopher's problem is considered a classic synchronization problem as it an example of a large class of concurrency control problems.

One simple solution is to represent each chop stick with a semaphore. A philosopher tries to grab a chopstick by executing a wait () operation on that semaphore; she releases her chop sticks by executing the signal () operation on the appropriate semaphores. Thus, the shared data are

Semaphore chopstick [5];

where all elements of chopstick are initialized to 1. This solution is rejected as it could create a dead lock. Suppose that all five philosophers become hungry simultaneously and each grabs her left chop stick. All the elements of chop stick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.

```
do {  
    wait(chopstick[i]);  
    wait(chopstick[(i+1) % 5]);  
    // eat  
    signal(chopstick[i]);  
    signal(chopstick[(i+1) % 5]);  
    // think  
} while (TRUE);
```

The structure of philosopher i .

Solution to dining philosophers problem:

- Allow at most four philosophers to be sitting simultaneously at the table
- Allow a philosopher to pick up her chopsticks only if both chopsticks are available.
- Use an asymmetric solution; that is, an odd philosopher picks up first her left chopstick and then her right chopstick whereas an even philosopher picks up her right chopstick and then her left chopstick.

MONITORS

While using semaphores we may have some errors like---

- * Suppose that a process interchanges the order in which the wait () and signal () operations on the semaphore mutex are executed.

```
Signal (mutex);  
.....  
Critical section  
.....  
Wait (mutex);
```

Here several processes may be executing in their critical sections simultaneously, violating the mutual exclusion requirement.

- * Suppose that a process replaces signal (mutex) with wait (mutex) that is it executes

```
Wait(mutex);  
.....  
Critical section  
.....  
Wait(mutex);
```

Here a deadlock will occur.

- * Suppose that a process omits the wait(mutex), or the signal(mutex) or both. Here, either mutual exclusion is violated or a dead lock will occur.

To deal with such errors, a fundamental high level synchronization construct called **monitor** type is used.

Usage

The monitor type also contains the declaration of variables whose values define the state of an instance of that type, along with the bodies of the procedures or functions that operate on those variables. The representation of a monitor type cannot be used directly by the various processes. Thus, a procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters. The local variables of a monitor can be accessed by only the local procedures.

The structure of a monitor is :

```
monitor monitorname
{
    Procedure P1(...)
    { .....}

    Procedure P2(...)
    { .....}

    :
    :
    Procedure Pn(...)
    { .....}

    Initialization code(.....)
    { .....}
}
```

The monitor construct ensures that only one process at a time can be active within the monitor. But this monitor construct is not powerful for modeling some synchronization schemes. For this we need additional synchronization mechanisms. These mechanisms are provided by condition construct. The only operations that can be invoked on a condition variable are wait() and signal(). The operation `x.wait()` means that the process invoking this operation is suspended until another process invokes `x.signal()`. The `x.signal()` operation resumes exactly one suspended process.

When `x.signal()` operation is invoked by a process P, there is a suspended process Q associated with condition x. If suspended process Q is allowed to resume its execution, the signaling process P must wait. Otherwise, both P and Q would be active simultaneously within the monitor

Dining Philosophers Solution using Monitors

This solution imposes the restriction that a philosopher may pick up her chopsticks only if both of them are available. To code this solution, we need to distinguish among three states in which we may find a philosopher. For this purpose, we use this data structure

```
enum {thinking, hungry, eating } state[5];
```

Philosopher i can set the variable $state[i] = \text{eating}$ only if her two neighbors are not eating: $(state[(i+4) \% 5] \neq \text{eating})$ and $(state[(i+1)\%5] \neq \text{eating})$

Also declare **condition** self [5];

where philosopher i can delay herself when she is hungry but is unable to obtain the chop sticks she needs. The distribution of the chopsticks is controlled by the monitor dp . Each philosopher before starting to eat, must invoke the operation $\text{pickup}()$. This may result in the suspension of the philosopher process. After the successful completion of the operation the philosopher may eat. Following this, the philosopher invokes the $\text{putdown}()$ operation. Thus philosopher i must invoke the operations $\text{pickup}()$ and $\text{putdown}()$ in the following sequence:

Monitor solution to Dining Philosophers problem

```
monitor DP
{
    enum { THINKING, HUNGRY, EATING } state [5];
    condition self [5];
    void pickup (int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING) self[i].wait;
    }
    void putdown (int i) {
        state[i] = THINKING;
        // test left and right neighbors
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}

void test (int i) {
    if ( (state[(i + 4) % 5] != EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] != EATING) ) {
        state[i] = EATING ;
        self[i].signal ();
    }
}

initialization_code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
```

Resuming Processes Within a Monitor

If several processes are suspended on condition x, and an x.signal() operation is executed by some process, then for determining which suspended process should be resumed next, we use FCFS ordering so that the process waiting the longest is resumed first. Or conditional wait construct() can be used as –

x.wait(c);

where c is an integer expression that is evaluated when the wait() operation is executed. The value of c which is called a **priority number** is then stored with the name of the process that is suspended. When x.signal() is executed, the process with the smallest associated priority number is resumed next.

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;

    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }

    void release() {
        busy = FALSE;
        x.signal();
    }

    initialization_code() {
        busy = FALSE;
    }
}
```

A monitor to allocate a single resource.

DEADLOCKS

In a multi programming environment, several processes may compete for a finite number of resources. A process requests resources and if the resources are not available at that time, the process enters a waiting state. Sometimes, a waiting process is never again able to change state because the resources it has requested are held by other waiting processes. This situation is called a **dead lock**.

SYSTEM MODEL:

A system consists of a finite number of resources to be distributed among a number of competing processes. The resources are partitioned into several types, each consisting of some number of identical instances. Memory space, CPU cycles, files and I/O devices are examples of resource types.

If a process requests an instance of a resource type, the allocation of any instance of the type will satisfy the request. If it will not, then the instances are not identical and the resource type classes have not been defined properly.

A process must request a resource before using it and must release the resource after using it. A process may request as many resources as it requires for carrying out its designated task.

Under the normal mode of operation, a process may utilize a resource in the following sequence:

1. Request – requesting process must wait until it can acquire the resource
2. Use – process can operate on the resource
3. Release – process releases the resource

The request and release of resources are system calls. Examples are the request () and release () device, open () and close () file and allocate () and free () memory system calls. Request and release of resources that are not managed by the OS can be accomplished through the wait () and signal () operations on semaphores or through acquisition () and release of mutex lock. A **system table** records whether each resource is free or allocated; for each resource that is allocated, the table also records the process to which it is allocated. If a process requests a resource that is currently allocated to another process, it can be added to a queue of processes waiting for this resource.

A set of processes is in a deadlock state when every process in the set is waiting for an event that can be caused only by another process in the set. The events mainly concerned here are resource acquisition and release. The resources may be either physical resources or logical resources.

Example 1: consider in a system there are three CD RW drives and there are three processes. Suppose each process holds one of these CD RW drives. If each process requests another drive, then the three processes will be in a deadlocked state. Each process is waiting for the event "CD RW released".

Example 2: consider in a system there is one printer and one DVD drive. Suppose that each process P_i is holding the DVD drive and process P_j is holding the printer. If P_i requests the printer and P_j requests the DVD drive, then a deadlock occurs.

DEADLOCK CHARACTERIZATION:

In a deadlock, processes never finish executing and system resources are tied up, preventing other jobs from starting.

Necessary Conditions

A deadlock situation can arise if following four conditions hold simultaneously in a system:

- * **Mutual exclusion:** At least one resource must be held in a non sharable mode; that is only one process at a time can use the resource.
- * **Hold and wait:** A process must be holding at least one resource and waiting to acquire additional resources that are currently being held by other processes.
- * **No preemption:** Resources cannot be preempted; that is a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- * **Circular Wait:** A set $\{P_0, P_1, \dots, P_n\}$ of waiting processes must exist such that P_0 is waiting for a resource held by $P_1, \dots, P_n$ is waiting for a resource held by P_0 .

All the above four conditions must hold for a deadlock to occur.

Resource Allocation Graph

Deadlocks can be described in terms of a directed graph called a **system resource allocation graph**. This graph consists of a set of vertices V and set of edges E . The set of vertices V is partitioned into two different types of nodes:

P –the set consisting of all the active processes in the system and

R –the set consisting of all resource types in the system.

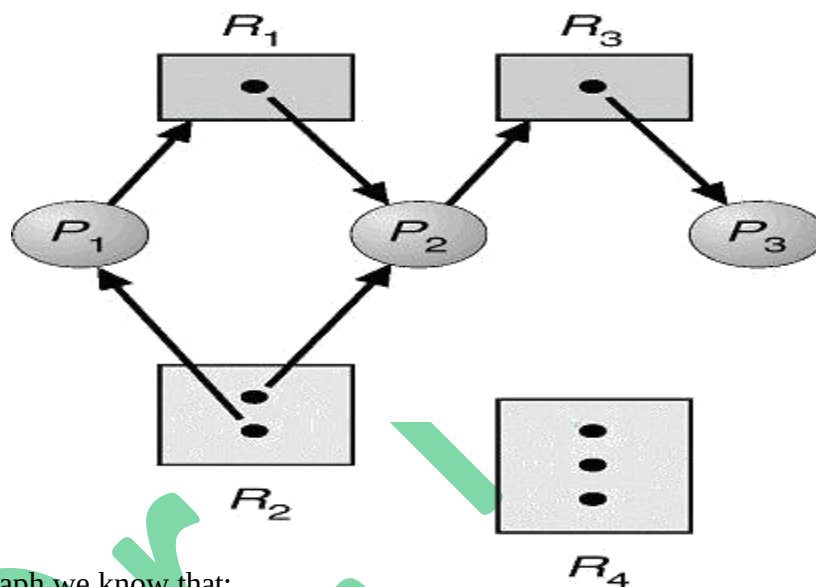
A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$; it signifies that process P_i has requested an instance of resource type R_j and is currently waiting for that resource.

A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$; it signifies that an instance of resource type R_j has been allocated to process P_i . A directed edge $P_i \rightarrow R_j$ is called a **request edge**. A directed edge $R_j \rightarrow P_i$ is called an **assignment edge**.

Process is represented using a circle and resource type is represented using a rectangle. Since resource type may have more than one instance, each instance is represented using a dot within the rectangle.

When a process requests an instance of resource type, a request edge is inserted in the resource allocation graph. When this request can be fulfilled, the request edge is instantaneously transformed to an assignment edge. When the process no longer needs access to the resource, it releases the resource; as a result, the assignment edge is deleted.

Example:



From this graph we know that:

- 1) The sets P, R and E :
 $P = \{p_1, p_2, p_3\}$
 $R = \{R_1, R_2, R_3, R_4\}$
 $E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$
- 2) Resource instances:
 There is one instance of type R_1
 There are two instances of type R_2
 There is one instance of type R_3
 There are three instances of type R_4
- 3) Process states:
 - a. Process P_1 is holding an instance of type R_2 and is waiting for an instance of type R_1 .
 - b. Process P_2 is holding an instance of type R_1 and is waiting for an instance of type R_3 .
 - c. Process P_3 is holding an instance of type R_3 .

If the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist. If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked. Here, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. Here, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

If a resource allocation graph does not have a cycle, then the system is not in a deadlocked state. If there is a cycle, then the system may or may not be in a deadlocked state.

Example:

In the above graph suppose that process P3 requests an instance of type R2. Since no resource instance is currently available, a request edge P3→R2 is added to the graph. At this point, there are two cycles in the graph.

That are:

P1 → R1 → P2 → R3 → P3 → R2 → P1

P2 → R3 → P3 → R2 → P2

Therefore, in this graph there is a cycle, so a deadlock occurs.

METHODS FOR HANDLING DEADLOCKS:

Deadlock problem can be dealt with in one of three ways –

- * Use a protocol to prevent or avoid deadlocks ensuring that the system will never enter a deadlock state
- * Allow the system to enter a deadlock state, detect it and recover
- * Ignore the problem altogether and pretend that deadlocks never occur in the system.

To ensure that deadlocks never occur, the system can use either deadlock prevention or a deadlock avoidance scheme. Deadlock prevention provides a set of methods for ensuring that at least one of the necessary conditions (listed under deadlock characterization) cannot hold. These methods prevent deadlocks by constraining how requests for resources can be made.

Deadlock avoidance requires that the OS be given in advance additional information concerning which resources a process will request and use during its lifetime. With this additional knowledge it can decide for each request whether or not the process should wait. To decide whether the current request can be satisfied or must be delayed, the system must consider the resources currently available, the resources currently allocated to each process and the future requests and releases of each process.

If a system does not employ either deadlock prevention or a deadlock avoidance algorithm, then a deadlock situation may arise. In this environment, the system can provide an algorithm that examines that state of the system to determine whether a deadlock has occurred and an algorithm to recover from the deadlock.

If a system neither ensures that a deadlock will never occur nor provides a mechanism for deadlock detection and recovery, then a situation arises where the system is in a deadlocked state yet has no way of recognizing what has happened. The undetected deadlock will result in deterioration of system's performance because resources are being held by processes that cannot run and because more and more processes as they make requests for resources will enter a deadlocked state. Eventually, the system will stop functioning and will need to be restarted manually.

In many systems, deadlocks occur infrequently, thus this method is cheaper than the prevention, avoidance or detection and recovery methods. In some situations, a system is in a frozen state but is not in a deadlocked state.

DEADLOCK PREVENTION:

For a deadlock to occur, each of the four necessary conditions must hold. By ensuring that at least one of these conditions cannot hold, we can prevent the occurrence of a deadlock.

Mutual Exclusion

The mutual exclusion condition must hold for non sharable resources (printer). Sharable resources do not require mutually exclusive access and thus cannot be involved in a deadlock (read only file). Example for the non sharable resource is the printer , ie at a time the two users cannot access the printer. Example for the sharable resource is Read only file, ie multiple users can do read operation at a time.

Hold and Wait

To ensure that the hold and wait condition never occurs in the system, we must guarantee that whenever a process requests a resource, it does not hold any other resources.

One protocol that can be used requires each process to request and be allocated all its resources before it begins execution.

An alternative protocol allows a process to request resources only when it has none. A process may request some resources and use them. Before it can request any additional resources, it must release all the resources that it is currently allocated.

To illustrate the difference between these two protocols, we consider a process that copies data from a DVD drive to a file on the disk, sorts the file and then print the results on a printer. If all resources must be requested at the beginning of the process, then the process must initially request the DVD drive, Disk and Printer. The second protocol allows the process to request initially only the DVD drive and Disk. It copies from DVD drive to Disk and then releases both DVD drive and Disk. Then the process again requests the Disk and printer. After copying the disk file to the printer, it releases these two resources and terminates.

Both these protocols have two main disadvantages. First, resource utilization may be low since resources may be allocated but unused for a long period. Second, starvation is possible. A process that needs several popular resources may have to wait indefinitely because at least one of the resources that it needs is always allocated to some other process.

No Preemption

The third necessary condition for deadlocks is that there be no preemption of resources that have already been allocated. To ensure that this condition does not hold, we use the following protocol.

If a process is holding some resources and requests another resource that cannot be immediately allocated to it, then all resources currently being held are preempted i.e. these resources are implicitly released. The preempted resources are added to the list of resources for which the process is waiting. The process will be restarted only when it can regain its old resources as well as the new ones that it is requesting.

Alternatively, if a process requests some resources, first check whether they are available. If they are, allocate them. If they are not, check whether they are allocated to some other process that is waiting for additional resources. If so, preempt the desired resources from the waiting process and allocate them to the requesting process. If the resources are neither available nor held by a waiting process, the requesting process must wait. While it is waiting, some of its resources may be preempted, but only if another process requests them. A process can be restarted only when it is allocated the new resources it is requesting and recovers any resources that were preempted while it was waiting. This protocol is applied to resources whose state can be easily saved and restored later such as CPU registers and memory space.

Circular Wait

The fourth and final condition for deadlocks is the circular wait condition. One way to ensure that this condition never holds is to impose a total ordering of all resource types and to require that each process requests resources in an increasing order of enumeration.

To illustrate, we let $R = \{R_1, R_2, \dots, R_m\}$ be the set of resource types. We assign a unique integer number to each resource type, which allows us to compare to resources and to determine whether one precedes another in our ordering. Formally, we define a one to one function $F: R \rightarrow N$, where N is the set of natural numbers. For example, if the set of resource types R includes tape drives, Disks and printers then the function F is defined as –

$F(\text{tape drive}) = 1$

$F(\text{disks}) = 5$

$F(\text{printer}) = 12$

Now we consider the following protocol to prevent dead locks: each process can request resources only in an increasing order. That is, a process can initially request any number of instances of a type R_i . After that the process can request instances of type R_j , if and only if $F(R_j) > F(R_i)$. If several instances of the same resourced type are needed, a single request for all of them must be issued.

DEADLOCK AVOIDANCE:

Deadlock prevention algorithms prevent deadlocks by restraining how requests can be made. The restraints ensure that at least one of the necessary conditions for deadlock cannot occur and hence that deadlocks cannot hold. Possible side effects of preventing deadlocks by this method are low device utilization and reduced system throughput.

An alternative method for avoiding deadlocks is to require additional information about how resources are to be requested.

The resource allocation state is defined by the number of available and allocated resources and the maximum demands of the processes.

Safe state

A state is **safe** if the system can allocate resources to each process (up to its maximum) in some order and still avoid a deadlock. A system is in a safe state only if there exists a **safe sequence**. If no such sequence exists, then the system state is said to be **unsafe**. A sequence of processes $\langle P_1, P_2, P_3, \dots, P_n \rangle$ is a safe sequence for the current allocation state if, for each P_i , the resource requests that P_i can still make can be satisfied by the currently available resources plus the resources held by all P_j , with $j < i$. Here if the resources that P_i needs are not immediately available, the P_i can wait until all P_j have finished. When they have finished, P_i can obtain all of its needed resources, complete its task, return its allocated resources and terminate. When P_i terminates, P_{i+1} can obtain its needed resources and so on.

A safe state is not a deadlocked state. A deadlocked state is an unsafe state. Not all unsafe states are deadlocks. An unsafe state may lead to a deadlock. As long as the state is safe, the OS can avoid unsafe (and deadlocked states). In an unsafe state, the OS cannot prevent processes from requesting resources such that a deadlock occurs. The behavior of the processes controls unsafe states.

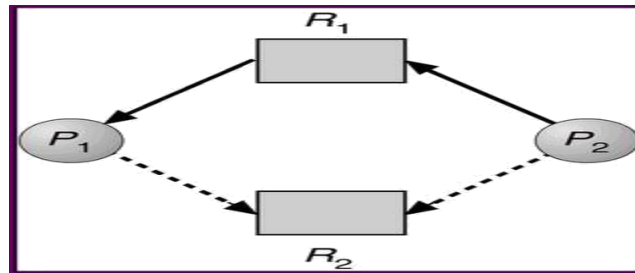
The idea is simply to ensure that the system will always remain in a safe state. Initially, the system is in a safe state. Whenever a process requests a resource that is currently available, the system must decide whether the resource can be allocated immediately or whether the process must wait. The request is granted only if the allocation leaves the system in a safe state.

Resource Allocation Graph Algorithm

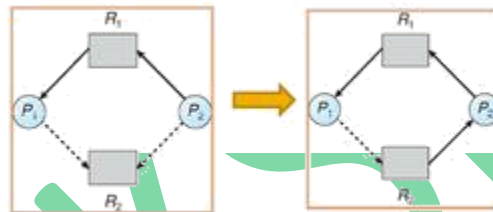
If there is a resource allocation system with only one instance of each resource type, a variant of the resource allocation graph can be used for deadlock avoidance. In addition to the request and assignment edges, a **claim edge** is also used. A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge resembles a request edge in direction but is represented in the graph by a dashed line. When a process P_i requests resource R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge. Similarly when resource R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.

If no cycle exists, then the allocation of the resource will leave the system in a safe state. If a cycle is found, then the allocation will put the system in an unsafe state.

Example:



Suppose that P_2 request R_2 . Although R_2 is currently available, we can not allocate it to P_2 , because this action will create a cycle in the graph. A cycle indicates that the system is in unsafe state. If P_1 requests R_2 and P_2 requests R_1 then a dead lock will occur.



Banker's Algorithm

The resource allocation graph algorithm is not applicable to a resource allocation system with multiple instances of each resource type. Banker's algorithm is applicable to such a system but is less efficient than the resource allocation graph scheme. The name was chosen because the algorithm could be used in a banking system to ensure that the bank never allocated its available cash in such a way that it could no longer satisfy the needs of all its customers.

Let n be the number of processes in the system and m be the number of resource types. The different data structures used in bankers algorithm are-

- * *Available*: vector of length m indicates the number of available resources of each type. If $available[j]=k$, there are k instances of type R_j available.
- * *Max*: an $n*m$ matrix defines the maximum demand of each process. If $Max[i][j]=k$, then process P_i may request maximum k instances of type R_j .
- * *Allocation*: an $n*m$ matrix defines the number of resources of each type currently allocated to each process. If $Allocation[i][j]=k$, then process P_i is currently allocated k instances of type R_j .
- * *Need*: an $n*m$ matrix indicates the remaining resource need of each process. If $need[i][j]=k$, then process P_i may need k more instances of type R_j .
 $Need[i][j] = Max[i][j] - Allocation[i][j]$.

Safety algorithm – algorithm for finding out whether or not a system is in a safe state.

1. Let Work and Finish be vectors of length m and n. initialize work = available and finish [i]= false.
 2. Find an i such that
 - a. Finish [i]== false
 - b. $Need_i \leq work$
- If no such i exists, go to step 4
3. $Work = work + Allocation_i$
Finish[i]=true.
Goto step 2.
 4. If finish[i] = true for all i, then the system is in a safe state

This may require an order of $m \cdot n^2$ operations to determine whether a state is safe.

Resource - Request Algorithm – algorithm which determines if requests can be safely granted.

Banker's Algorithm - Resource Request

Request_i - request vector - the number of additional instances for each resource type process i requests at this time.

1. If not ($Request_i \leq Need_i$) raise an error - process i tries to get more resources than what it declared.
2. If not ($Request_i \leq Available$) process i must wait - no sufficient resources at this time.
3. **Tentatively allocate** the requested resources to process i:
 $Available := Available - Request_i$
 $Allocation_i := Allocation_i + Request_i$
 $Need_i := Need_i - Request_i$
4. Check safety of state. If safe, the resources are **allocated**.
If not safe then **cancel the tentative allocation** and process i must wait.



Illustrative Example

Banker's Algorithm - Example

- 5 processes and 3 resource types A (with 10), B (with 5), and C (with 7 instances).
- A Snapshot:

Available				Allocation			Max		
A	B	C		A	B	C	A	B	C
3	3	2	P ₀	0	1	0	7	5	3
			P ₁	2	0	0	3	2	2
			P ₂	3	0	2	9	0	2
			P ₃	2	1	1	2	2	2
			P ₄	0	0	2	4	3	3

- Is this a safe state?

The matrix need is defined as max-Allocation and is as follows:

<u>Need</u>			
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

Banker's Algorithm - Example

- 5 processes and 3 resource types A (with 10), B (with 5), and C (with 7 instances).
- A Snapshot:

Available				Allocation			Max			Need		
A	B	C		A	B	C	A	B	C	A	B	C
3	3	2	P ₀	0	1	0	7	5	3	7	4	3
			P ₁	2	0	0	3	2	2	1	2	2
			P ₂	3	0	2	9	0	2	6	0	0
			P ₃	2	1	1	2	2	2	0	1	1
			P ₄	0	0	2	4	3	3	4	3	1

<P₁, P₃, P₀, P₂, P₄> is a safe sequence.

Banker's Algorithm - Example (cont.)

- Suppose P₁ now requests (1, 0, 2).

Available				Allocation			Max			Need		
A	B	C		A	B	C	A	B	C	A	B	C
2	3	0	P ₀	0	1	0	7	5	3	7	4	3
			P ₁	3	0	2	3	2	2	0	2	0
			P ₂	3	0	2	9	0	2	6	0	0
			P ₃	2	1	1	2	2	2	0	1	1
			P ₄	0	0	2	4	3	3	4	3	1

<P₁, P₃, P₀, P₂, P₄> is a safe sequence also in this case.

Here we find that there is a safe sequence <P₁,P₃,P₄,P₀,P₂>. So we can immediately grant the request of process P₁.

DEADLOCK DETECTION:

If a system does not employ either a deadlock prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. Here the system must provide:

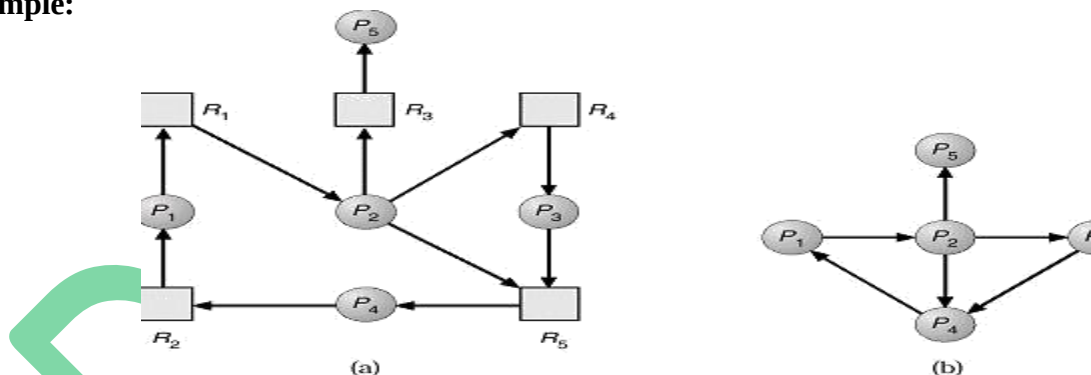
- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from deadlock.

Single instance of each resource type

If all resources have only a single instance, then we can define a deadlock detection algorithm that uses a variant of the resource allocation graph called a **wait for graph**. This graph is obtained from the resource allocation graph by removing the resource nodes and collapsing the appropriate edges.

An edge from P_i to P_j in a wait for graph implies that process P_i is waiting for process P_j to release a resource that P_i needs. An edge $P_i \rightarrow P_j$ exists in a wait for graph if and only if the corresponding resource allocation graph contains two edges $P_i \rightarrow R_q$ and $R_q \rightarrow P_j$ for some resource R_q .

Example:



A deadlock exists in the system if and only if the wait for graph contains a cycle. To detect deadlocks, the system needs to maintain the wait for graph and periodically invoke an algorithm that searches for a cycle in the graph. An algorithm to detect a cycle in the graph requires an order of n^2 operations where n is the number of vertices in the graph.

Several instances of a resource type

The wait for graph scheme is not applicable to resource allocation system with multiple instances of each resource type. For several instances of resource type, the algorithm employs several time varying data structures. They are:

- * Available – a vector of length m indicates the number of available resources of each type

- * Allocation – an $n \times m$ matrix defines the number of resources of each type currently allocated to each process
- * Request – an $n \times m$ matrix indicates the current request of each process.

Algorithm:

1. Let Work and Finish be vectors of length m and n . initialize work = available and if $allocation_i \neq 0$ then $finish[i] = false$, otherwise $finish[i] = true$.
2. Find an i such that
 - c. $Finish[i] == false$
 - d. $Request_i \leq work$
- If no such i exists, go to step 4
3. $Work = work + Allocation_i$
 $Finish[i] = true$.
 Goto step 2.
4. If $finish[i] = false$ for some i , then the system is in a dead locked state

This algorithm requires an order of $m \times n^2$ operations to detect whether the system is in a deadlocked state.

Detection algorithm usage

When should the detection algorithm be invoked? This depends on two factors:

- How often is a deadlock likely to occur?
- How many processes will be affected by deadlock when it happens?

If deadlocks occur frequently, then the detection algorithm should be invoked frequently. Resources allocated to deadlocked processes will be idle until the deadlock can be broken.

Deadlocks occur only when some process makes a request that cannot be granted immediately. This request may be the final request that completes a chain of waiting processes. The deadlock detection algorithm can be invoked every time a request for allocation cannot be granted immediately. But if the deadlock detection algorithm is invoked for every resource request, this will incur a considerable overhead in computation time.

RECOVERY FROM DEADLOCK:

When a detection algorithm determines that a deadlock exists, several alternatives are available. There are two options for breaking a deadlock. One is simply to abort one or more processes to break the circular wait. The other is to preempt some resources from one or more of the deadlocked processes.

Process Termination

To eliminate deadlocks by aborting a process, use one of the two methods. In both methods, the system reclaims all resources allocated to the terminated processes.



Abort all deadlocked processes – Breaks the deadlock cycle, the deadlocked processes may have computed for a long time and the results of partial computations must be discarded and will have to be recomputed later.



Abort one process at a time until the deadlock cycle is eliminated – Incurs considerable overhead, since after each process is aborted, a deadlock detection algorithm must be invoked to determine whether any processes are still deadlocked.

If the partial termination method is used, then we must determine which deadlocked process should be terminated. Abort those processes whose termination will incur minimum costs.

Many factors may affect which process is chosen including:



What the priority of the process is



How long the process has computed and how much longer the process will compute before completing its designated task



How many and what type of resources the process has used



How many more resources the process needs in order to complete



How many processes will need to be terminated



Whether the process is interactive or batch

Resource Preemption

To eliminate deadlocks using resource preemption, preempt some resources from processes and give these resources to other processes until the deadlock cycle is broken.

If preemption is required to deal with deadlocks, three issues need to be addressed:

- Selecting a victim – Which resources and which processes are to be preempted?
- Rollback – If a resource is preempted from a process, what should be done with that process?
- Starvation – How do we ensure that starvation will not occur?

In a system where victim selection is based primarily on cost factors, it may happen that the same process is always picked as a victim. So this process never completes its designated task, a starvation situation that must be dealt with in any practical system. A process can be picked as a victim only a finite number of times.

-----*****-----